New Algorithms for Role Hierarchy Construction and Evaluation on Mined Roles

Anonymous

Abstract

Role hierarchies in Role-Based Access Control (RBAC) enable scalable administration by allowing permissions to be inherited among roles. However, deriving high-quality hierarchies from existing flat RBAC policies remains a challenging combinatorial problem. We present two new algorithms for the *Role Hierarchy Mining Problem (RHMP)* that construct compact and well-structured hierarchies while preserving all user–permission authorizations. The first algorithm, *MinRolesRH*, restructures a minimal-role RBAC policy into a multi-layer hierarchy without introducing new roles, optimizing for maximal depth and minimal edge count. The second algorithm, *NewRolesRH*, generalizes this approach by introducing new intermediate roles to maximize hierarchical depth. To further minimize redundant edges, we formulate *Integer Linear Programs (ILPs)* that prune edges while preserving authorizations. We evaluate the resulting hierarchies using both *Weighted Structural Complexity (WSC)* and a *random-walk based navigability metric*, which estimates the expected traversal steps from users to permissions. Experimental results on standard benchmarks and RMPLib datasets demonstrate that our algorithms consistently achieve lower WSC and improved navigability compared to existing algorithms, including *RHMiner* and *Role Generation + Elimination*. All implementations are available as open source.

CCS Concepts

• **Mathematics of computing** → **Solvers**; • **Theory of computation** → **Graph algorithms analysis**; • **Security and privacy** → **Access control**.

Keywords

Role Mining, Role Hierarchy, Linear Programming

ACM Reference Format:
Anonymous. 2025. New Algorithms for Role Hierarchy Construction and Evaluation on Mined Roles. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (SACMAT '26).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Role mining is the problem of devising techniques to construct *roles* — intermediate entities through which authorized users gain access to permissions. It is a core component of the Role-Based Access Control (RBAC) model, which provides an abstraction between users and permissions to simplify security administration. In its standard form, RBAC policies are *flat*, meaning that roles do not inherit permissions from one another. However, real organizations often exhibit hierarchical structures in which higher-level roles inherit the privileges of lower-level ones. Extending RBAC to incorporate such inheritance leads to the *Role Hierarchy Mining Problem* [11], which aims to automatically derive a hierarchical RBAC configuration from existing access-control data.

In this paper, we propose two new algorithms for the Role Hierarchy Mining Problem and compare them to two existing baselines: *Role Generation + Elimination* [22] and *RHMiner* [11]. Our algorithms differ in how they construct the hierarchy. *Algorithm 1* restructures an existing RBAC policy without introducing new roles, optimizing for the maximal number of *layers* while minimizing the number of *edges*. *Algorithm 2* generalizes this approach by allowing the creation of new intermediate roles. In doing so it further maximizes the number of layers, but the cost is additional roles and edges introduced into the hierarchy. We demonstrate through our experiments on running our algorithms, that it is not always the case that additional roles and edges worsen the functional quality.

We evaluate the hierarchies produced by these algorithms using two complementary metrics. The first is the *Weighted Structural Complexity (WSC)*, which measures the compactness of the resulting structure. The second is a *Random-Walk based navigability metric*, which estimates the expected number of traversal steps needed for a user to reach all authorized permissions. Together, these measures capture both the structural and functional quality of the resulting hierarchies.

The renewed interest in this problem stems from the recent publication of publicly available benchmarks for role mining [1, 2]. These benchmarks have facilitated comparative evaluation of algorithms and have motivated new approaches to role mining. In particular, recent work has introduced algorithms that aim to minimize the number of roles [18] and the number of edges [9], both evaluated on these benchmark datasets.

We present a motivating example in Figure 1, which compares three configurations: (a) an RBAC policy minimizing the number of roles (Figure 1a), (b) an RBAC policy minimizing the number of edges (Figure 1b), and (c) a role hierarchy (Figure 1c) derived from these policies.

One can derive the hierarchy in 1c from 1b in a straightforward way. The permissions of role 3 $\{p_0, p_1, p_2\}$ are a superset of those of roles 1 $\{p_0, p_1\}$ and 2 $\{p_1, p_2\}$, allowing us to define role 3 as a senior role inheriting from them. The resulting hierarchy has only twelve edges, fewer than in the original flat policy. In this, there are no new roles introduced in the hierarchy.

Similarly, when deriving from 1a, we observe a non-empty intersection $\{u_2, u_3, u_4, u_5\}$ among the users assigned to the two roles in 1a. This suggests that introducing a new intermediate role serving

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SACMAT '26, Waterloo, Canada
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

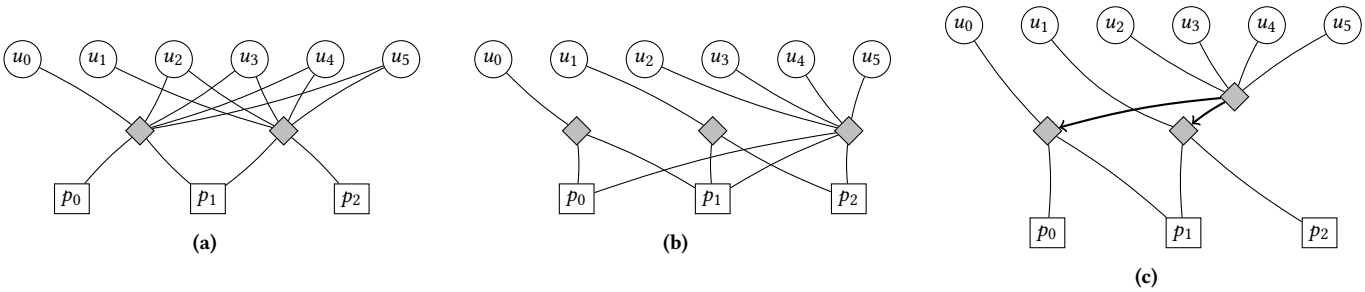


Figure 1: An RBAC policy for the access matrix that minimizes the number of roles (a). An RBAC policy that minimizes the number of edges (b). A Role-Hierarchy shown in (c) which can be derived from policies in (a) or (b)

these users can further simplify the structure. With this introduction of a new role, and the restructuring, which involves removing edges from $\{u_2, u_3, u_4, u_5\}$ to the two existing roles and assigning the new role to the existing roles, yields the hierarchy in 1c.

Our contributions. We contribute to the Role Hierarchy Mining Problem (RHMP) line of work by proposing two algorithms: (i) a *restructuring* method that constructs a hierarchy without introducing new roles and optimizes the hierarchical depth, followed by edge pruning to remove redundancy; and (ii) a *role-introducing* method that adds intermediate roles to further increase hierarchical depth. To evaluate hierarchy quality, we use the Weighted Structural Complexity (WSC) metric for structural compactness, hierarchical depth for vertical richness, and complement these with a random walk based navigability metric. For baselines, we compare against our implementations of *Role Generation + Elimination* in [22] and *RHMiner* [11] as representative prior approaches.

Experimental setup. All our experiments were run on a 64-core 4023S-TRT rack-mounted server with 256 GB RAM. The machine runs Ubuntu 24.04 LTS operating system, the code we have written is in Python 3 and our constraint-solver is Gurobi version 12.

Organization. The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces background on the Role Hierarchy Mining Problem (RHMP) and defines the measures of hierarchy quality used in our evaluation. Section 4 describes two representative prior approaches and our implementations of them as baselines. Section 5 presents our two new algorithms, and Section 6 reports the experimental results and evaluates their performance on benchmark datasets.

2 Related Work

The problem of Role Mining was first introduced in the paper by Vaidya et al. in [20]. Other variants of the Role-Mining problem exist with the objective of minimizing the number of edges and have been studied in many works [8, 9, 16, 19].

Later, in [11] Guo et. al introduce two variants of the problems related to Role-Hierarchies, *a*) *Role-Hierarchy Building Problem* (RHBP), *b*) *Role-Hierarchy Mining Problem* (RHMP). In RHBP a set of roles is given and objective is to generate a Role-Hierarchy with minimal number of edges. On the other hand, in RHMP the objective is to construct a directed acyclic graph of roles from an existing RBAC policy such that the number of roles and edges in the graph is minimal.

Role mining (flat RBAC).. To our knowledge, the role–mining problem was first articulated by Vaidya et al. [20], with extensive surveys and taxonomies appearing subsequently. Beyond minimizing the number of roles, a distinct line of work targets *minimizing the number of edges*, the number of user–role and role–permission assignments, often termed *edge concentration*. Ene et al. in [8] proposed a greedy construction paired with an edge reduction heuristics. In [8] the authors introduced benchmark instances which were later used in many subsequent works [9, 14, 18, 22]. Vaidya et al. in [19] formulated the edge objective (Edge_RMP) and gave greedy/ILP formulations. A recent reassessment implements and compares these approaches, where the *edge concentration* approach in [8] is used as a strong baseline, and introduces new algorithms for edge minimization [9]. Minimizing the *number of roles* is qualitatively different from minimizing edges; recent work leverages maximal biclique enumeration both to obtain exact solutions on many instances and to produce strong heuristics where exact search is infeasible [18].

Role hierarchies. Moving from flat RBAC to hierarchical RBAC, Molloy et al. connected role discovery to Formal Concept Analysis (FCA), yielding *HierarchicalMiner* and introducing *Weighted Structural Complexity (WSC)* as a general structural cost for comparing RBAC (and hierarchical) policies. Guo, Vaidya, and Atluri formalized the *Role Hierarchy Building Problem* (RHBP) where the objective is to construct a complete hierarchy over a given role set with the fewest edges and the more general *Role Hierarchy Mining Problem* (RHMP) — derive roles and a hierarchy from a user–permission relation to minimize roles plus edges [11]. These formulations clarify that hierarchy quality hinges on both *graph structure* and *authorization preservation*.

Dataset. Ene et al. [8] introduce a polynomial-time algorithm to remove what they call *dominator edges* from input instances when solving for a minimum-role policy. In their work, they also present a heuristic algorithm for the edge-concentration problem. They experiment on ten benchmark instances that have since become standard in subsequent studies [9, 14, 15, 18, 22]. We use these same benchmarks to evaluate our implementations of prior algorithms and to compare them with our new approaches.

In addition, Anderer et al. [2] propose a new benchmark suite for role mining that includes both synthetic datasets and instances derived from real-world scenarios. We use this dataset as well to further test the scalability and robustness of our algorithms.

3 Background

In this section, we provide definitions that are relevant to Role-Hierarchies and the problem of mining them.

The Role Hierarchy Mining Problem. In literature [11], the Role Hierarchy Mining Problem (RHMP) is defined as following.

Definition 1 (Role Hierarchy Mining Problem (RHMP)). *Given a set of users U, a set of permissions P and a user-permission assignment UP, find a set of roles R, a user-role assignment UA, a role-permission assignment PA and role-role assignment RH, such that $RH = G(V, E)$ is consistent with UP that minimizes the $|R| + |E|$*

An observation in the above definition is that this problem as defined in [11] minimizes the role-role edges only. In our work, we generate hierarchies with distinct roles, where a role is defined to distinct in the following way.

Definition 2. *A role r in the hierarchy is distinct if the $users(r) \cup perms(r)$ does not equal to the users and permissions of another role in the hierarchy.*

Definition 3 (Strict RH). *Given an access control configuration $\rho = \langle \mathcal{U}, \mathcal{P}, UP \rangle$ and an integer $L > 0$ where $\mathcal{U}$ is the set of users, $\mathcal{P}$ is the set of permissions and $UP \subseteq \mathcal{U} \times \mathcal{P}$ is the user-permission assignment, we define the Role-Hierarchy Mining Problem as the following:*

The goal is to finding an RBAC state $\Gamma = \langle \mathcal{R}, UA, PA, RH \rangle$, where:

- $\mathcal{R} = \{\mathcal{R}_1, \ldots, \mathcal{R}_L\}$*: collection of sets of distinct roles*
- $UA \subseteq \mathcal{U} \times R_L$*: assign roles at layer L to users*
- $PA \subseteq \mathcal{R}_1 \times \mathcal{P}$*: assign roles at layer 1 to permissions*
- $RH \subseteq \bigcup_{i=1}^{L-1} \mathcal{R}_i \times \mathcal{R}_{i+1}$*: role hierarchy, encoding the inheritance among roles*

The structure of the hierarchy is as follows:

- *Roles in $\mathcal{R}_L$ are assigned to users and roles in $\mathcal{R}_{L-1}$.*
- *Roles in $\mathcal{R}_1$ are assigned to permissions and roles in $\mathcal{R}_2$.*
- *Intermediate roles in $\mathcal{R}_2, \ldots, \mathcal{R}_{L-1}$ are assigned to roles in both adjacent layers.*

The resulting configuration must preserve the original user-permission mapping, and the objective of this problem is to maximize the number of layers.

In our work, Algorithm 2, *NewRolesRH* generates hierarchies which follow Definition 3.

3.1 Measures of Quality

When mining for roles, several measures of quality can be used to assess an RBAC policy. The most common objectives optimize either the number of roles, the number of edges, or their weighted sum, as in [11]. In all these measures, the notion of *minimality* is central to achieving a compact and manageable access-control configuration.

A frequently used quality criterion for assessing Role Hierarchies is the *Weighted Structural Complexity (WSC)* [15]. This measure provides a unified framework for quantifying structural compactness as a linear combination of the number of roles, user–role, role–role, and role–permission edges, as well as optional direct user–permission assignments.

Definition 4 (Weighted Structural Complexity). *Given a weight vector $W = (w_r, w_u, w_p, w_h, w_d)$, where each weight $w_i \in \mathbb{Q}^+ \cup \{\infty\}$, the weighted structural complexity $wsc(\gamma, W)$ of an RBAC configuration $\gamma = (ROLES, UA, PA, RH, DUPA)$ is defined as:*

$$wsc(\gamma, W) = w_r|R| + w_u|UA| + w_p|PA| + w_h|t_{reduce}(RH)| + w_d|DUPA|,$$

where $t_{reduce}(RH)$ denotes the transitive reduction of the role hierarchy RH.

In this work, we use the Weighted Structural Complexity as a primary structural quality metric for comparing generated role hierarchies. We disallow direct user–permission assignments, and therefore adopt the weight vector $W = (1, 1, 1, 1, 0)$.

3.2 Navigability Measure via Random Walks

Beyond structural compactness, a desirable role hierarchy should also be *navigable*, allowing efficient propagation of authorizations from users to their permissions. In graph-theoretic terms, the *hitting time* $HIT(u, v)$ of a random walk is the expected number of steps required to reach a vertex v starting from vertex u [13]. We define the total navigability cost of a hierarchy as the average expected hitting time between all users and their authorized permissions.

Random walks have been widely studied in biology, physics, and computer science to model diffusion and transport phenomena on networks [6, 17]. In computer science, beyond their theoretical importance for understanding graph structure [13], random walks serve as a foundation for well-known applications such as the *PageRank* algorithm for measuring node importance [5], graph-embedding methods like *Node2Vec* [10], and navigability analysis in multilayer transportation systems [7].

Intuitively, hierarchies with fewer redundant edges tend to exhibit lower expected hitting times, indicating easier traversal between users and permissions. This intuition parallels that of biased random walks in Node2Vec, which generate low-dimensional embeddings that capture structural relationships in a graph. We adopt this random-walk based measure to complement the Weighted Structural Complexity (WSC), jointly capturing both the *structural compactness* and the *functional accessibility* of mined role hierarchies.

The metric also reflects the efficiency of access-enforcement mechanisms: in practice, enforcement algorithms perform structured traversals—though not random ones—to determine the permissions a user is authorized for. Hence, a more navigable structure is desirable.

Formally, we define navigability score of a role hierarchy RH as:

$$nav(RH) = \mathop{\mathbb{E}}_{u \in \text{users}(RH)} \left[\mathbb{E} \left[\sum_{p \in \text{perms}(u)} HIT(u, p) \right] \right]$$

Here:

- $\text{users}(RH)$ is the set of users in the role hierarchy RH.
- $\text{perms}(u)$ is the set of permissions authorized to user u.

- trials denotes repeated Monte-Carlo random-walk simulations approximating expected hitting times.
- $HIT(u, p)$ returns the random-walk hitting time from u to p.

A hierarchy with a smaller *nav* score is desirable as it is more accessible. Authorizations propagate between users and their permissions with fewer traversal steps.

Algorithm 1: `randomWalk`

```
Input  : RH: Role Hierarchy
foreach curr ∈ users(RH) do
    Get permissions reachable from curr in the RH, call it
      targets;
    steps ← 0;
    while not all targets visited do
        curr ← Pick a random neighbour of curr;
        Increment steps;
        if curr ∈ targets then
            Mark curr as visited;
score ← average of steps for each user;
return score;
```

We use the algorithm 1 to perform the random walk on the role hierarchy. The algorithm starts from a user and picks a random neighbour it has visited all permissions authorized to the user. In doing so, it increments *steps* each time it chooses a random node. To compute a score for the entire hierarchy, it finds the average of *steps* for each user. In our experiments, we perform five trials of the random walk on each role hierarchy and take the average of the *score* from all trials.

Since performing random walks on large graphs can be computationally intensive, in our experiments we employ a random sampling technique to estimate the *nav score*. We determine the required sample size n for a specified confidence level $(1 - \alpha)$ and margin of error ϵ following standard statistical methods [4]. In this technique, we randomly sample n users and compute the *hitting times* for those users and use that to estimate the *nav score* for the hierarchy.

4 Prior Approaches

We implement two existing algorithms *RHMiner* [11] and *Role Generation + Elimination* [22] from prior works. We are unaware of any publicly available implementations of these algorithms, and have implemented them ourselves; these implementations are part of our open source offering [3].

4.0.1 RHMiner. The *RHMiner* algorithm decomposes the Role Hierarchy Mining Problem into two phases: (1) derive an RBAC policy with a minimal number of roles, and (2) apply the *RHBuilder* algorithm [11] to construct a hierarchy with the minimal number of edges. In this approach, each role is represented as a subset of permissions. The *RHBuilder* algorithm incrementally builds the hierarchy by examining containment relationships between roles. If a role r is *contained* in another role s (i.e., $perms(r) \subseteq perms(s)$), then s inherits r. During this process, the algorithm greedily removes redundant edges to maintain a minimal transitive structure.

The construction begins by introducing a *super-role* sr that incorporates all permissions present in the user–permission assignment (UPA). The super-role serves as a universal ancestor, ensuring that every role in the hierarchy is connected by at least one path from sr. Initially, an edge (sr, r) is added for each role r in the flat RBAC policy. Each role r is then compared with every direct descendant r_i of sr to determine the relationship between their permission sets.

If the permissions of r and r_i are disjoint, the subtree rooted at r_i is ignored, since r cannot be related to any of its descendants. If the permissions of r *contain* those of r_i ($perms(r_i) \subseteq perms(r)$), an edge (r, r_i) is added, establishing r as a senior role of r_i. In this case, the direct link (sr, r_i) is removed and replaced by the indirect path (sr, r) and (r, r_i). Any redundant edges from r to permissions, users, or descendant roles of r_i are also removed, since these relationships are now reachable through r_i.

Conversely, if the permissions of r_i contain those of r ($perms(r) \subseteq perms(r_i)$), then (sr, r) is removed, as r is already reachable through r_i. The algorithm then recursively applies the same procedure within the subtree rooted at r_i to maintain the minimal inheritance structure. Finally, when r and r_i have overlapping but non-containment permission sets, *RHBuilder* searches within the subtree of r_i for any descendant r_j whose permissions are fully contained within $perms(r)$. If such a descendant is found, the edge (r, r_j) is added and any redundant edges between r and descendants of r_j are removed. This ensures that each inheritance edge in the resulting hierarchy is both necessary and non-redundant.

Initial RBAC policy. For all our implementations, we chose the policy with minimal number of roles initially and then build the hierarchies out of those. It was not always possible to generate policies with least number of roles in a reasonable amount of time. We follow the work of Tripunitara et al. [18] to generate these policies. We classify the benchmark inputs into two categories: *easy* and *hard*. The instances are hard if producing RBAC policy with minimum number of roles takes longer than 1 day. Out of 57 inputs, there are 12 hard inputs. We discuss them in Section 6.1.

In Table 1, we present the performance of *RHMiner* on the dataset. It displays the number of roles, layers and edges in the generated hierarchy. The table shows the data for *easy* inputs where the number of layers are at least two. Out of the 45 *easy* inputs, there are 26 inputs where the *RHMiner* yielded some changes to the initial RBAC policy. There is one input `rw 01` with 4 layers and 6 inputs with 3 layers and the remaining 19 have 2 layers.

4.0.2 Role Generation + Elimination. In [22], Xu et al. introduce algorithms for role mining that optimize for various policy-quality metrics, including one based on the *Weighted Structural Complexity (WSC)* and another on the interpretability of roles with respect to user attributes. The algorithm for generating role hierarchies consists of two main components: the *Role Generation* and *Elimination* phases.

The *Role Generation* algorithm first constructs an initial set of roles, where each role contains all permissions assigned to a single user. It then derives additional candidate roles by computing all unique intersections of permission sets from these initial roles, as described in CompleteMiner [20]. From these candidate roles, a role hierarchy is created by introducing inheritance edges between roles whose permission sets have subset / superset relationships.

Table 1: Results of Role Hierarchy Construction after running the *RHMiner* on *easy* instances

Benchmark	#Roles	Layers	#Edges
fw1	64	3	2185
fw2	10	2	1222
hc	14	3	199
domino	20	2	729
apj	453	3	4189
as	178	3	8802
al	398	3	89156
univ	18	2	717
mailer	565	3	5373
small 02	25	2	393
small 03	25	2	416
small 05	49	2	533
small 06	50	2	740
small 07	34	2	1254
medium 03	199	2	5961
medium 06	250	2	7542
large 03	499	2	7569
large 06	500	2	11260
comp 01.1	400	2	9533
comp 01.2	400	2	10859
rw 01	463	4	356509
2level 01	15	2	468
2level 03	25	2	512
2level 04	25	2	768
2level 09	50	2	1058
2level 10	50	2	1587

The *Elimination* phase removes roles from this candidate hierarchy if doing so preserves consistency with the original user-permission assignments and improves policy quality. In our implementation, we use WSC as the policy-quality metric optimized by the elimination phase.

In our experiments, the algorithm did not perform well in practice, as it failed to scale to larger instances and often timed out after running for one day. On one smaller instance, domino, it produced a hierarchy with the best WSC of 465; however, on the remaining 23 out of 57 instances where it completed, it yielded worse results than our newer approaches. Hence, we omit it from our main results.

5 Our Work

In this section, we present our work – our two algorithms for hierarchical role mining.

5.1 Algorithm 1: MinRolesRH

We present our first algorithm, *MinRolesRH*, which begins with an RBAC policy that minimizes the number of roles and restructures it into a multi-layered role hierarchy.

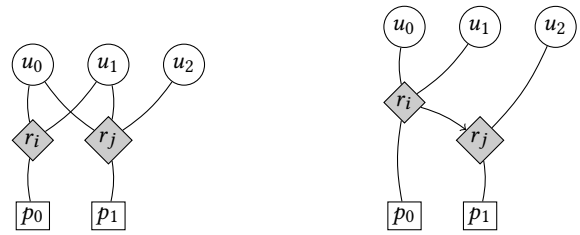
Given a minimum-role RBAC policy where each role has an associated set of users and permissions, we can view each role either as the set of users assigned to it or as the set of permissions it grants. Under either view, an ordering can be defined on a subset of roles based on set containment.

As the first step in our approach, we compute a minimum-role RBAC policy using the maximal biclique enumeration algorithm proposed in [18]. From this policy, we extract the sets of users and permissions assigned to each role. We then derive two ordered sequences of roles:

- A *user chain* $r_1, r_2, \ldots, r_k$ such that $users(r_1) \subset users(r_2) \subset \ldots \subset users(r_k)$.
  This sequence is computed using Algorithm 2, where we construct a create a directed acyclic graph (DAG) in which vertices are roles and two vertices u, v have a directed edge between them if users of u are a subset of users assigned to v. A chain is the longest path from a vertex in this DAG.
- A *permission chain* $r'_1, r'_2, \ldots, r'_k$ such that $perms(r'_1) \subset \ldots \subset perms(r'_k)$.
  The logic for computing this sequence is analogous to that of Algorithm 2, but performs a subset containment for permissions of roles instead of users.

These ordered sequences are then used to construct the role hierarchy.

Figure 2a illustrates an example where role r_i is assigned users $\{u_1, u_2\}$ and permission p_1, while role r_j is assigned users $\{u_1, u_2, u_3\}$ and permission p_2. Here, $users(r_i) \subset users(r_j)$, which allows r_j to be placed as the child of r_i to form the hierarchy shown in Figure 2b. Although users of r_i have access to both p_1 and p_2, the roles in the original RBAC policy are not maximal bicliques. Hence, a two-layer hierarchy can represent the same access structure more compactly.

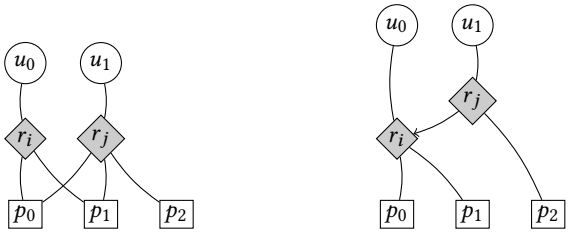


(a) RBAC policy **(b) Role Hierarchy**

Figure 2: Example showing user subsets in RBAC policy in 2a converting to Role Hierarchy in 2b

A similar example is shown in Figure 3a, where $perms(r_i) \subset perms(r_j)$. The corresponding two-layer hierarchy is illustrated in Figure 3b, where r_j inherits r_i.

To put it together, in Algorithm 3, we have the procedure that restructures an RBAC policy into a hierarchy. The algorithm first computes the user and permission chains from a policy π, it then selects either the user or the permission chains, preferring the one that contains the longest chain. This ensures that the final hierarchy achieves the maximum possible hierarchical depth of roles. When the user chains are selected, we iterate over each chain and make every pair of consecutive roles in the chain, say r_i and r_j, inherit one another according to their order. Specifically, in user chains, the predecessor role r_i inherits the successor role r_j.



(a) RBAC policy **(b) Role Hierarchy**

Figure 3: Example showing permission subsets in RBAC policy in 3a converting to Role Hierarchy in 3b

Algorithm 2: Algorithm for getting user chains in a policy

Input : π: RBAC policy

Create a graph $G = (V, E)$ where each vertex is a role;
foreach $r_i \in V$ **do**
  foreach $r_j \in V \setminus \{r_i\}$ **do**
    if $users(r_i) \subset users(r_j)$ **then**
      Add a directed edge (r_i, r_j);
foreach $r \in V$ **do**
  Perform a depth-first search traversal on G starting at r;
  $chain \leftarrow$ track the vertices visited in path from r during traversal until the path cannot be extended;
return *all such chains*;

After establishing the inheritance relationships, we update the users directly assigned to the roles in the chains prune redundant direct user–role assignments.

Similarly, for permission chains, we perform the same steps except that the direction of inheritance in each chain is reversed. Finally, we update the permissions assigned to each role in the hierarchy to remove redundant direct permission assignments. Some redundant inheritance edges may still remain, which are removed in our edge-minimization step described in Section 5.2. By Claim 1, the hierarchy produced by this algorithm achieves the maximum hierarchical depth when no new roles are introduced. This is validated by our experimental results, which show that the hierarchies produced by our algorithm have a greater number of layers than those produced by *RHMiner*.

5.2 Minimizing Number of Edges

After the hierarchy is constructed, it may still contain redundant edges. In this section, we discuss the post-processing strategies used to minimize the number of edges in the role hierarchies produced by the *MinRolesRH* algorithm. In our experiments, these approaches are applied in combination.

5.2.1 Prune role–role edges. First, we introduce a pruning procedure to remove redundant role–role edges in the hierarchy. A role–role edge is considered *redundant* if a role is both connected to another role directly and indirectly, via some other role. To eliminate such redundant edges, we apply Algorithm 4. For each role r, we collect its direct children S_r and compute the set I_r of nodes

Algorithm 3: Algorithm for `MinRolesRH`(π)

Input: RBAC policy, π

get `user-chains` using Algorithm 2;
get `permission-chains`;
Create an empty hierarchy RH, mapping each role to its associated entities $\{e : e \in users \cup permissions \cup roles\}$;
if *the longest chain is from* `user-chains` **then**
  foreach `chain` $\in$ `user-chains` **do**
    foreach *consecutive roles* r_i, r_j *in* `chain` **do**
      assign r_j as a child of r_i in RH;
  foreach *role* $r \in RH$ **do**
    assign users of r from π to r in RH;
  foreach `chain` $\in$ `user-chains` **do**
    for $i \in [2, |chain|]$ **do**
      $r_i, r_{i-1} \leftarrow$ roles at index i and $(i-1)$;
      assign only $users(r_i) \setminus users(r_{i-1})$ to r_i in RH;
else
  foreach `chain` $\in$ `permission-chains` **do**
    foreach *consecutive roles* r_i, r_j *in* `chain` **do**
      assign r_i as a child of r_j in RH;
  foreach $r \in RH$ **do**
    assign permissions of r from π to r in RH;
  foreach `chain` $\in$ `permission-chains` **do**
    for $i \in [2, |chain|]$ **do**
      $r_i, r_{i-1} \leftarrow$ roles at index i and $(i-1)$;
      assign only $perms(r_i) \setminus perms(r_{i-1})$ to r_i in RH;
return RH;

reachable from r via paths of length at least two. This reachability set I_r is obtained using a depth-first search (DFS) traversal over the hierarchy.

Any direct child role $s \in S_r$ that also appears in I_r is reachable from r through another path, making the edge (r, s) redundant, hence it can be removed. This yields a transitive reduction of the hierarchy, eliminating redundant role–role edges.

Algorithm 4: Algorithm for removing role–role edges

Input : Role hierarchy $RH = (R, E)$

foreach *role* $r \in R$ **do**
  get direct descendant roles of r, call it $S_r \leftarrow \{ s \mid (r, s) \in E,\ s \in R \}$;
  get descendant roles of r reachable via length ≥ 2 paths, let it be I_r;
  foreach $s \in S_r \cap I_r$ **do**
    remove edge (r, s) from E ;

5.2.2 ILP construction for removing redundant user–role and role–permission edges. Following the pruning of role–role edges, we propose two ILP-based pruning procedures.

The first, presented in this section, removes redundant user–role and role–permission edges from a hierarchy while preserving authorizations. The second, described in Section 5.2.3, focuses on

path-based pruning between users and permissions, but does not scale well on large inputs when used alone. Therefore, we introduce this complementary procedure to eliminate as many redundant edges as possible before constructing the path-based ILP. The ILP formulation in this section can be viewed as a special case of the path-based ILP, and for smaller inputs, the path-based formulation alone is often sufficient.

We formulate the Integer Linear Program (ILP) for this pruning procedure as follows:

Notation. Let U, R, and P denote the sets of users, roles, and permissions, respectively. Let $\mathsf{UP} \subseteq U \times P$ denote the user–permission assignment. Let RH represent the role hierarchy. For each pair $(u, p) \in \mathsf{UP}$, define a set of *entry* and *exit* roles where each entry role is connected to u and exit role is connected to p:

$$\Pi_{u,p} = \{(e, x) \mid e, x \in R, \text{ and there exists a path } e \rightsquigarrow x \text{ in } RH, \text{ and } (u, e), (x, p) \text{ are edges in } RH\}$$

Each pair $(e, x) \in \Pi_{u,p}$ represents a possible *entry* and *exit* role through which user u can reach permission p.

Decision variables. We define three types of binary variables,

- $x_{u,r} \in \{0, 1\}$, whether user u is assigned to role r.
- $y_{r,p} \in \{0, 1\}$, whether role r is assigned permission p.
- $z_{u,e,x,p} \in \{0, 1\}$, if paths from $(e \rightsquigarrow x)$ in the hierarchy is used to connect user u to permission p where $(e, x) \in \Pi_{u,p}$ are entry and exit role pairs.

Motivating our approach, in Figure 4, a user u and its *entry roles* e_1, e_2 are shown. The entry roles are the roles where u is directly connected to in the hierarchy. It also has a permission p and its *exit roles* x_1, x_2, x_3. If u is authorized to p in the UP, there exists a path $u \rightsquigarrow p$ and every such path has to use one of the entry roles and one of the exit roles. We can develop constraints so that there is at least one path between u to p in the hierarchy and it uses the least number of entry and exit roles.

Constraints. We have the following constraints.

- There exists at least one path connecting u to p that uses at least one of the entry and exit roles.
- If either $x_{u,e} = 0$ or $y_{x,p} = 0$, then there is no path $u \rightsquigarrow p$ using entry role e and exit role x.
- If u is not assigned to r in the hierarchy then $x_{u,r} = 0$.
- If p is not assigned to r in the hierarchy then $y_{r,p} = 0$.

Objective. : the objective is to minimize the number of user–role and role–permission edge variables.

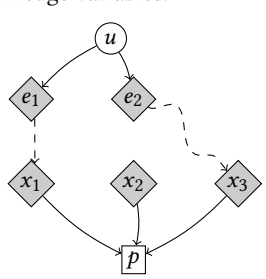


Figure 4: User u with its entry roles e_1, e_2 and permission p with exit roles x_1, x_2, x_3

The key in this construction is that, we do not enumerate all paths between a user and permission but rather only check if they are connected via some path.

5.2.3 Integer Linear Program (ILP) construction for removing redundant edges in the Role-Hierarchy. We introduce an ILP formulation that minimizes the number of edges in the role hierarchy. The formulation specifies the notation, variables, constraints, and objective necessary to ensure that all user–permission authorizations remain intact while redundant edges are eliminated.

The intuition is that if it is possible to compute all paths between all users and their authorized permissions, we can ensure that an edge on a path is not redundant if no user uses that edge to get to one its authorized permissions.

Notation. Let U, R, and P denote the sets of users, roles, and permissions, respectively. Let $\mathsf{UP} \subseteq U \times P$ denote the user–permission assignment. Let RH represent the role hierarchy. For each pair $(u, p) \in UP$, define a set of *entry* and *exit* roles where each entry role is connected to u and exit role is connected to p:

$$\Pi_{u,p} = \{(e, x) \mid e, x \in R, \text{ and } (u, e), (x, p) \text{ are edges in } RH\}$$

Each pair $(e, x) \in \Pi_{u,p}$ represents a possible *entry* and *exit* role through which user u can reach permission p. For this construction, we enumerate all paths starting from u to p.

Decision variables. We define four types of binary variables.

- $x_{u,e} \in \{0, 1\}$, an edge variable describing if user u is assigned to entry role e.
- $y_{x,p} \in \{0, 1\}$, an edge variable describing if exit role x is assigned permission p.
- $z_{r,s} \in \{0, 1\}$, an edge variable describing if role r is assigned role s.
- $path_{u,p,i} \in \{0, 1\}$, if path i is used to go from user u to permission p.

Constraints. To minimize the number of edges in the hierarchy, we will need the following constraints:

- If all edges on a path are active, then this path may be used.
- There exists at least one path connecting u to p.

Objective. The objective is to minimize all *edge* variables.

We present a complete mathematical formulation of the ILPs in Appendix D.

Algorithm 5: Algorithm to minimize edges in subgraphs

Input :Role hierarchy $RH = (R, E)$, k

Sample a subgraph from the RH of size k;
Construct and solve the ILP 2;
Remove user–role edges where $x_{u,r} = 0$;
Remove permission–role edges where $y_{r,p} = 0$;
Remove role–role edges where $z_{r,s} = 0$;

In practice, this ILP formulation does not scale well, as it requires exploring all possible paths between users and permissions within the hierarchy. For large hierarchies, this leads to *path explosion*. However, for moderately sized graphs, this approach yields hierarchies with the smallest number of edges among all our tested

methods. For large graphs, we restrict the number of paths between a user and its authorized permission to at most 10.

To address scalability further, we decompose the edge-minimization problem into three stages:

(1) pruning redundant role–role edges (Section 5.2.1)
(2) pruning redundant user–role and role–permission edges (Section 5.2.2),
(3) repeatedly running Algorithm 5 on subgraphs of the role hierarchy of size k.

In our experiments, we execute Algorithm 5 repeatedly on overlapping subgraphs. The intuition is that pruning edges on smaller subgraphs and repeatedly stitching the resulting pruned hierarchies back together can eliminate almost all redundant edges. The number of iterations used is set to $10 \cdot k^2 (\log |E_R| + 1)$, where k is the number of vertices in each subgraph and E_R is the set of redundant edges that can be removed from the hierarchy. Since there are at least $|U| + |P|$ edges that are not redundant, we can upper bound $|E_R|$ by $|E| - |U| - |P|$ where E are the edges in the hierarchy. The derivation of this bound is provided in Appendix C.

5.3 Algorithm 2: `NewRolesRH`

This is our second algorithm, in which we begin with a candidate RBAC policy and introduce new roles that can be added to higher layers of the hierarchy. The goal is to construct a hierarchy with the *maximum possible number of layers*. Introducing new roles ensures that each role remains distinct.

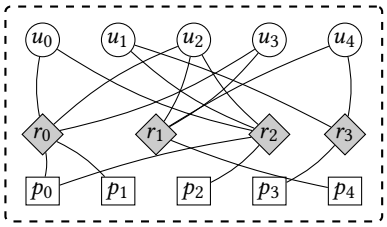


Figure 5: Initial RBAC policy

We begin with an example RBAC policy shown in Figure 5. It has 5 users, 4 roles, and 5 permissions. Each role has users and permissions assigned to it. User u_0 is assigned to roles r_0 and r_1, and gains permissions $\{p_0, p_1, p_2\}$ via these roles. User u_1 is assigned to r_2 and r_3, and gains $\{p_0, p_2, p_3\}$. User u_2 is assigned to r_0, r_1, and r_2, and gains $\{p_0, p_1, p_2, p_4\}$. User u_3 gains $\{p_0, p_1, p_4\}$ via r_0 and r_1, while u_4 gains $\{p_3, p_4\}$ via r_1 and r_3. All these roles are distinct because no two roles share the same set of users and permissions.

The question, then, is whether a new layer of roles can be created while maintaining this distinctness. In this example, a role in a new layer can be created if it inherits permissions from *at least two roles* in the layer below. Inheriting from multiple roles guarantees distinctness, since each new role represents a unique combination of permissions.

In Figure 6, role r_4 inherits from r_0 and r_2, and users previously assigned to r_0 and r_2 can now also be assigned to r_4. Similarly, r_5 inherits from r_2 and r_3, and users assigned to both r_2 and r_3 are now assigned to r_5. Following the same principle, r_6 and r_7 can be created.

Set-Cover Interpretation. Another way to interpret the creation of a new role layer is through the *set-cover* perspective. For each user, the set of permissions they are authorized to corresponds to the universe of elements that must be covered. Each role acts as a subset of these elements. Identifying a set of roles that together cover a user's permission set corresponds to forming a new role.

For example, consider user u_2, who requires $\{p_0, p_1, p_2, p_4\}$ and is initially assigned to roles r_0, r_1, and r_2. The permissions of these roles are: $r_0 \rightarrow \{p_0, p_1\}$, $r_1 \rightarrow \{p_4\}$, $r_2 \rightarrow \{p_0, p_2\}$. The smallest cover requires all three roles: $\{r_0, r_1, r_2\}$. If we constrain each new role to inherit from *exactly two roles*, we can maximize the number of layers in the hierarchy. Under this constraint, we can form up to three new roles: (r_0, r_1), (r_0, r_2), (r_1, r_2). Two of these can then be used to cover u_2's permissions. In Figure 6, u_2 is assigned to r_4 and r_7, where r_4 inherits from (r_0, r_2) and r_7 inherits from (r_0, r_1). Since the smallest cover requires three roles, as shown in Figure 7, a new layer can be created in which r_8 inherits from r_4 and r_7.

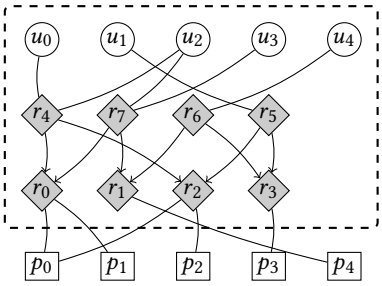


Figure 6: Role Hierarchy with one additional layer

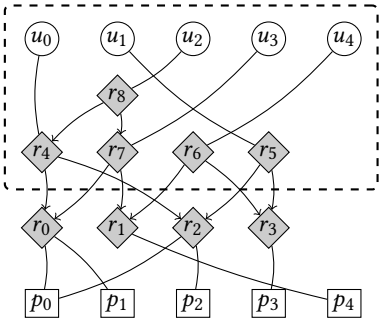


Figure 7: Role Hierarchy with two additional layers

Set Cover to ILP.. We use the standard reduction from the Set Cover problem to an Integer Linear Program (ILP) [21], which we solve using *Gurobi* [12]. In our setting, the universe corresponds to the set of permissions assigned to a user, and the subsets correspond to the roles available in the top-most layer of the hierarchy.

In Algorithm 6, we construct a role hierarchy with the maximal possible number of layers from an input RBAC policy π. We begin by initializing the hierarchy RH to be π itself. The algorithm then iteratively creates new layers of roles until no additional roles can be generated. A new role is created if it can cover the set of permissions required by a user, and the corresponding users are assigned to it.

Algorithm 6: `NewRolesRH`

Input : π: RBAC policy
initialize the role hierarchy RH to be π;
while *True* **do**
  foreach $u \in UP$ **do**
    identify the set of roles in π that cover $perms(u)$, denote this set by R_u;
    create a new role r' that inherits from roles in R_u;
    assign the users $\bigcup_{r \in R_u} users(r)$ to r';
  if *no new roles were created* **then**
    break;
  remove redundant roles within the current layer and reassign their users accordingly;
  add the newly created roles as a new layer in RH;
  update UP so that each role in the new top layer considers the roles in the layer immediately below as its permission;
return RH;

After each iteration, redundant roles—those for which all assigned users can already be covered by other roles in the same layer—are removed. When a new layer is added to the hierarchy, the roles in the second-highest layer become the permission sources for the roles in the top-most layer, and the algorithm repeats until no further distinct roles can be formed.

5.4 Results for *MinRolesRH* and *NewRolesRH*

In Figure 8, we present the percentage decrease in the number of roles following edge pruning for *MinRolesRH* and *NewRolesRH*. There was no significant decrease in the number of edges for *MinRolesRH*. For all *easy* inputs shown in the figure, the decrease in edges was below 20%. This was not the case for *NewRolesRH*, where many instances had reductions exceeding 50%. The exceptions are `emea`, `al`, `apj`, `domino`, and `rw 01`.

We observed that *NewRolesRH* introduced at least twice as many roles as *MinRolesRH* in 31 out of 57 inputs. There were two instances–`emea` and `comp 03.1`–where no new roles were introduced. In terms of edges, four instances–`fw1`, `fw2`, `as`, and `univ`–had fewer edges than the hierarchy produced by *MinRolesRH*, and `emea` and `comp 03.3` had no additional edges. Only `comp 02.1–02.3` and `comp 04.1–04.3` exhibited more than twice the number of edges as in *MinRolesRH*; all remaining inputs had edge counts within a factor of two.

6 Evaluation and Results

In this section, we present the results of our experiments and provide key observations. We use the *RHMiner* algorithm as the baseline and compare our algorithms to it. We compare the hierarchies based on their hierarchical depth, structural compactness and navigability.

Hierarchical Depth. When comparing the algorithms on hierarchical depth, as shown in Figure 9, we observe that *NewRolesRH* yields hierarchies with most depth for all inputs. In the figure, we exclude data for `comp 02.4, comp 04.4` as the number of layers in hierarchies produced by `NewRolesRH` is 120 and 594 respectively, while *RHMiner* did not introduce any layers and *MinRolesRH* introduced 3 and 5 respectively. This comes at a cost of additional roles being added to the hierarchy. On the other hand, *MinRolesRH* yields hierarchies with depth at least as good as the baseline, *RHMiner* without incurring additional cost of roles.

Structural Compactness and Navigability. In Table 3 in Appendix A, we present the WSC and *nav* scores for the *easy* instances. Each section of the table contains results for the three algorithms described in this paper. The first section reports the results for *RHMiner* described in Section 4.0.1, followed by *MinRolesRH* from Section 5.1, and finally *NewRolesRH* from Section 5.3. Each section in the table has two columns: one for the WSC of the hierarchy and another for the *nav* score.

In 39 out of 45 instances, *MinRolesRH* produces hierarchies with the lowest WSC, indicating that the hierarchy is the most structurally compact. There are eight instances where *RHMiner* has the same WSC as *MinRolesRH*, and for all these instances, the number of layers is also equal. There are two instances: `mailer` and `2level 09`, where *RHMiner* yields the hierarchy with the lowest WSC. When compared in terms of the *nav* score, in these two instances, *MinRolesRH* performs better than *RHMiner*, meaning that despite being structurally less compact, the navigability of the hierarchy actually improved. Moreover, when the edges in the hierarchy produced by *RHMiner* were pruned using our approach, the WSC further decreased to 5897 for `mailer` and 1029 for `2level 09`. Interestingly, there are 4 instances where *NewRolesRH* yields hierarchies with the lowest WSC. Despite having higher number of roles and layers in `fw1`, `fw2`, and `univ`, the hierarchies have fewer edges compared to those produced by the other algorithms.

In terms of navigability, *NewRolesRH* yielded hierarchies with the lowest *nav* score in 13 instances, suggesting that increased structural complexity does not necessarily hinder navigability.

Performance. In our experiments, the algorithms we implemented in this paper, all ran within 1 day. We utilize approaches mentioned to construct RBAC policies in [18], the time to construct the policies were similar to the times mentioned in their work.

6.1 Addressing Hard Instances

For benchmark inputs `comp 02.1–02.4`, `comp 03.1–03.4`, and `comp 04.1–04.4`, we do not enumerate all maximal bicliques when computing the RBAC policy. This is because, with a large number of maximal bicliques, generating an RBAC policy with the minimal number of roles becomes computationally infeasible within a reasonable amount of time. In our experiments, we set the threshold on the number of maximal bicliques to 5 million. We therefore classify these inputs as "hard" instances.

For `comp 02.1–02.3` and `comp 04.1–04.3`, we follow the approach of Tripunitara *et al.* [18]. Specifically, we identify bicliques above a certain threshold size—large enough and create roles for those bicliques. This procedure reduces the size of the remaining input, allowing us to subsequently apply the maximal biclique enumeration algorithm on the remaining input. Among these inputs, `comp 03.1–03.4` are the hardest instances. Our system does not

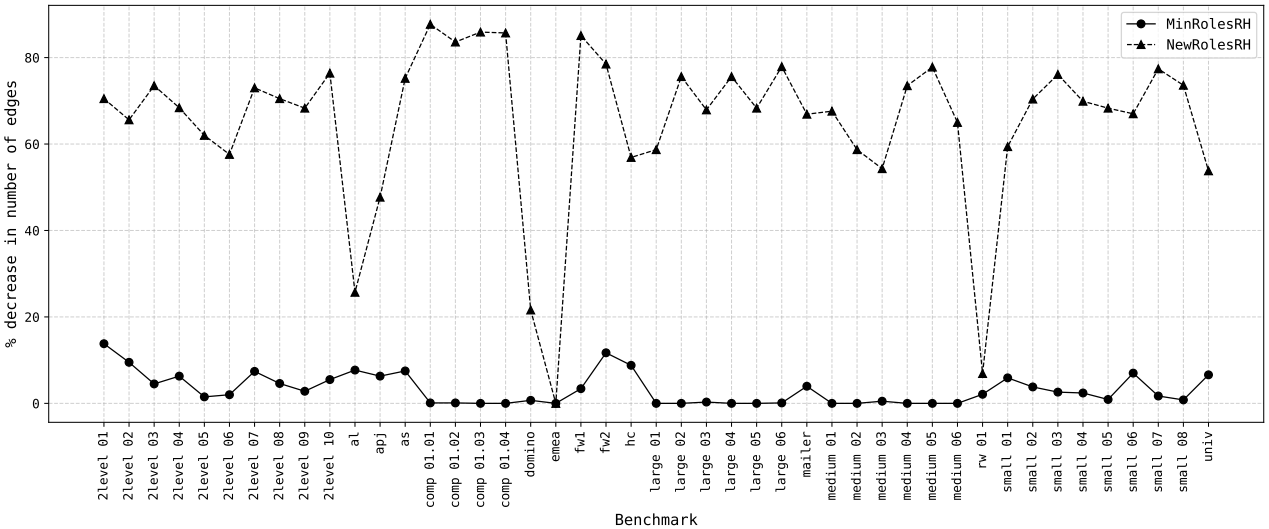


Figure 8: Percentage decrease in the number of edges: *MinRolesRH, NewRolesRH*

have sufficient memory to compute the full adjacency information required for maximal biclique enumeration. Instead, for the hardest instances and `comp 02.4`, `comp 04.4` we use the greedy and lattice-based post-processing algorithm described in [8, 18] to generate the initial RBAC policy.

Table 2: WSC and Random Walk scores for role hierarchies generated by *RHMiner* [11], *MinRolesRH, NewRolesRH* on *hard* isntances

Benchmark	*RHMiner*		*MinRolesRH*		*NewRolesRH*	
	WSC	***nav* score**	**WSC**	***nav* score**	**WSC**	***nav* score**
comp 02.1	64164	310411	64158	271220	170386	430382
comp 02.2	94360	349188	91732	326355	207656	174046
comp 02.3	118537	281719	117177	274929	258633	329324
comp 02.4	463907	301930	462746	301070	530893	123096
comp 03.1	394889	234974	358797	91849	484488	356901
comp 03.2	743916	104281	712859	95704	713436	107500
comp 03.3	887988	207679	549082	137230	549082	289359
comp 03.4	1207722	140417	1165196	134194	1189441	127575
comp 04.1	127782	543143	120011	598777	327004	308805
comp 04.2	140009	694812	141368	669360	365024	248486
comp 04.3	149719	593962	149698	591235	405797	331057
comp 04.4	165799	654626	142206	691045	256812	566318

Table 2 presents the WSC and *nav* scores for the hard instances. For 11 out of 12 instances, *MinRolesRH* achieves the lowest WSC, and in 5 of these, it also attains the lowest *nav* score. For `comp 03.3`, both of our algorithms produced the lowest WSC, and running *NewRolesRH* did not lead to any improvement in the hierarchical depth. There are 7 instances where *NewRolesRH* yields the lowest *nav* score despite having higher WSC compared to *RHMiner* and *MinRolesRH*.

For `comp 04.2` where *RHMiner* has the lowest WSC, with the pruning step, it was possible to bring it further down to 140004. For `comp 02.4, comp 04.4` having large number of layers introduced by *NewRolesRH* assisted in navigability of the hierarchy.

7 Conclusion

We presented two algorithms – *MinRolesRH* and *NewRolesRH*, for constructing compact and navigable role hierarchies from mined RBAC policies. Our methods balance minimizing structural complexity, measured by Weighted Structural Complexity (WSC), with maximizing hierarchical depth and navigability. Across a diverse set of benchmark instances, *MinRolesRH* consistently produced hierarchies with the low WSC and strong navigability. *NewRolesRH*, while introducing additional roles often yielded deeper hierarchies with good navigability. We further demonstrated that our edge-pruning post-processing step effectively reduces redundancy and can improve hierarchies generated by other algorithms as well.

Future Work. There is considerable scope for future work. One direction is to study the navigability metric in greater depth, including how it affects access enforcement algorithms and how it can be incorporated into role mining and role hierarchy mining problems as an optimization objective. Another direction is to investigate the importance of individual roles within the hierarchy. Existing work [22] on the interpretability of roles can be extended in this context. A further line of work could explore ways to introduce separation of duty by incorporating mutually exclusive roles within hierarchies.

References

[1] Simon Anderer. 2020. RMPlib. https://github.com/RMPlib/RMPlib

[2] Simon Anderer, Bernd Scheuermann, Sanaz Mostaghim, Patrick Bauerle, and Matthias Beil. 2021. RMPlib: A Library of Benchmarks for the Role Mining Problem. In *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies* (Virtual Event, Spain) *(SACMAT '21)*. Association for Computing Machinery, New York, NY, USA, 3–13. doi:10.1145/3450569.3463566

[3] Anonymous. 2025. Role Hierarchies. https://github.com/anonresearch2205/role-hierarchies

[4] Emmanuel N Barron and John G Del Greco. 2022. *Probability and Statistics for STEM*. Springer.

[5] Fan Chung. 2014. A Brief Survey of PageRank Algorithms. *IEEE Trans. Netw. Sci. Eng.* 1, 1 (2014), 38–42.

[6] Edward A Codling, Michael J Plank, and Simon Benhamou. 2008. Random walk models in biology. *Journal of The Royal Society Interface* 5, 25 (2008), 813–834. arXiv:https://royalsocietypublishing.org/doi/pdf/10.1098/rsif.2008.0014 doi:10.1098/rsif.2008.0014

[7] Manlio De Domenico, Albert Solé-Ribalta, Sergio Gómez, and Alex Arenas. 2014. Navigability of interconnected networks under random failures. *Proceedings of the National Academy of Sciences* 111, 23 (2014), 8351–8356. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.1318469111 doi:10.1073/pnas.1318469111

[8] Alina Ene, William Horne, Nikola Milosavljevic, Prasad Rao, Robert Schreiber, and Robert E Tarjan. 2008. Fast exact and heuristic methods for role minimization problems. In *Proceedings of the 13th ACM symposium on Access control models and technologies*. 1–10.

[9] Puneet Gill and Mahesh Tripunitara. 2025. New Algorithms for Minimizing the Number of Edges in Bottom-Up Role Mining. In *Proceedings of the 30th ACM Symposium on Access Control Models and Technologies* (USA) *(SACMAT '25)*. Association for Computing Machinery, New York, NY, USA, 22–32. doi:10.1145/3734436.3734443

[10] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.

[11] Qi Guo, Jaideep Vaidya, and Vijayalakshmi Atluri. 2008. The Role Hierarchy Mining Problem: Discovery of Optimal Role Hierarchies. In *Proceedings of the 2008 Annual Computer Security Applications Conference (ACSAC '08)*. IEEE Computer Society, USA, 237–246. doi:10.1109/ACSAC.2008.38

[12] LLC Gurobi Optimization. 2025. Gurobi. https://www.gurobi.com/

[13] László Lovász. 1993. Random walks on graphs. *Combinatorics, Paul erdos is eighty* 2, 1-46 (1993), 4.

[14] Ziqing Mao, Qihua Wang, and Jorge Lobo. 2009. Evaluating role mining algorithms. (2009).

[15] Ian Molloy, Hong Chen, Tiancheng Li, Qihua Wang, Ninghui Li, Elisa Bertino, Seraphin Calo, and Jorge Lobo. 2008. Mining roles with semantic meanings. In *Proceedings of the 13th ACM symposium on Access control models and technologies*. 21–30.

[16] F. J. Newbery. 1989. Edge concentration: a method for clustering directed graphs. In *Proceedings of the 2nd International Workshop on Software Configuration Management* (Princeton, New Jersey, USA) *(SCM '89)*. Association for Computing Machinery, New York, NY, USA, 76–85. doi:10.1145/72910.73350

[17] James P Sethna. 2021. *Statistical mechanics: entropy, order parameters, and complexity*. Vol. 14. Oxford University Press.

[18] Mahesh Tripunitara. 2025. Minimizing the Number of Roles in Bottom-Up Role-Mining using Maximal Biclique Enumeration *(SACMAT '25)*. Association for Computing Machinery, New York, NY, USA, 4–15. doi:10.1145/3734436.3734442

[19] Jaideep Vaidya, Vijayalakshmi Atluri, Qi Guo, and Haibing Lu. 2009. Edge-RMP: Minimizing administrative assignments for role-based access control. *J. Comput. Secur.* 17, 2 (April 2009), 211–235.

[20] Jaideep Vaidya, Vijayalakshmi Atluri, and Janice Warner. 2006. RoleMiner: mining roles using subset enumeration. In *Proceedings of the 13th ACM Conference on Computer and Communications Security* (Alexandria, Virginia, USA) *(CCS '06)*. Association for Computing Machinery, New York, NY, USA, 144–153. doi:10.1145/1180405.1180424

[21] Vijay V Vazirani. 2001. *Approximation algorithms*. Vol. 1. Springer.

[22] Zhongyuan Xu and Scott D Stoller. 2012. Algorithms for mining meaningful roles. In *Proceedings of the 17th ACM symposium on Access Control Models and Technologies*. 57–66.

8 Appendices

A Results

In Figure 9, we present the number of layers in hierarchies produced by each algorithm and Table 3 describes the WSC and *nav* scores for *easy* benchmark inputs.

Table 3: WSC and Random Walk scores for role hierarchies generated by *RHMiner* [11], *MinRolesRH*, *NewRolesRH*

Benchmark	*RHMiner*		*MinRolesRH*		*NewRolesRH*	
	WSC	***nav* score**	**WSC**	***nav* score**	**WSC**	***nav* score**
fw1	2249	19570	2275	12285	2085	6854
fw2	1232	10200	1033	8101	996	8329
hc	213	1224	212	684	411	880
domino	749	2326	742	1392	782	1232
apj	4642	5284	4519	2932	4854	3077
as	8980	58053	10088	20258	9436	20270
al	89554	545580	88355	218723	92766	231836
emea	7280	48256	7280	48256	7280	48256
univ	735	3177	700	1991	692	1876
mailer	5938	20783	6024	17029	6412	12589
small 01	301	960	278	772	326	814
small 02	418	1467	403	1090	449	1135
small 03	441	1677	431	1468	490	1491
small 04	523	2316	511	2129	555	2216
small 05	582	1742	579	1551	678	1655
small 06	790	3004	772	2526	864	2548
small 07	1288	6964	1280	6163	1333	6404
small 08	1069	4985	1061	4356	1134	4733
medium 01	3269	11309	3268	10180	3571	10616
medium 02	5367	22749	5367	22749	5521	23072
medium 03	6160	34242	6130	30897	6311	30945
medium 04	4651	23725	4651	23725	5085	24952
medium 05	6717	24611	6716	23001	7203	23964
medium 06	7792	44223	7792	42151	8079	42401
large 01	8927	39863	8927	39863	9326	40603
large 02	10613	22071	10612	21981	11742	23828
large 03	8068	28429	8059	27450	9179	28209
large 04	11403	70918	11403	70918	12341	71386
large 05	16385	40813	16385	40813	16905	36150
large 06	11760	79333	11760	68110	12961	69760
comp 01.01	9933	49569	9921	47796	10570	49748
comp 01.02	11259	57377	11249	57235	11866	57785
comp 01.03	10483	51269	10479	49950	11088	50779
comp 01.04	11810	59218	11807	58768	12513	59505
rw 01	356972	297534	355277	224456	384368	139271
2level 01	483	1510	422	1262	447	1034
2level 02	500	800	445	707	487	716
2level 03	537	1909	514	1600	558	1783
2level 04	793	1746	781	1503	793	1496
2level 05	755	2447	744	2202	787	2167
2level 06	769	2575	754	2326	793	2292
2level 07	1096	4117	1017	3301	1041	3515
2level 08	969	2166	915	1959	980	1949
2level 09	1108	3012	1147	2814	1243	2752
2level 10	1637	4537	1540	3904	1674	4240

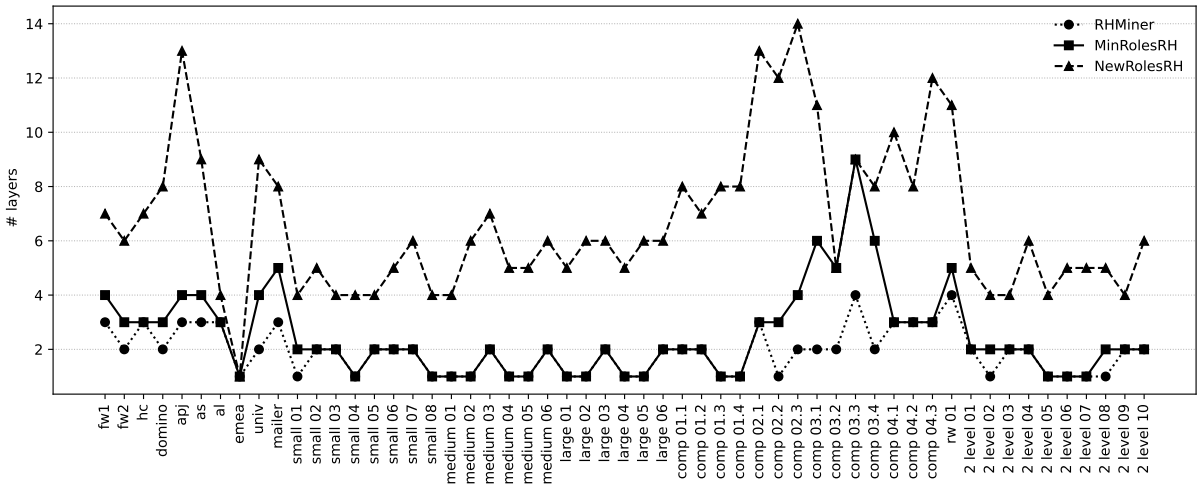
B MinRolesRH

Claim 1. *Given an RBAC policy π with n roles $\{R_1, R_2, \ldots, R_n\}$, the maximum layers in a Role-Hierarchy where no new roles are to introduced is bounded by the maximum of largest user-chain and the permission-chain.*

Suppose:

- *L_1 is the longest sequence of roles such that $perms(R_{i_1}) \subset perms(R_{i_2}) \subset \cdots \subset perms(R_{i_{L_1}})$*

Figure 9: Number of layers in the role hierarchy produced by each Algorithm: *RHMiner, MinRolesRH, NewRolesRH*



- *L_2 is the longest sequence of roles such that $users(R_{j_1}) \subset users(R_{j_2}) \subset \cdots \subset users(R_{j_{L_2}})$*

Then the maximum number of layers is $L = \max(L_1, L_2)$

Proof. By contradiction. Assume that the longest sequence of roles $L = [R_{i_1}, R_{i_2}, \ldots, R_{i_L}]$ in the Role-Hierarchy RH is neither L_1 nor L_2. In L, each role is assigned to its successor. That is, R_{i_1} is assigned to R_{i_2}, R_{i_2} is assigned to R_{i_3} and so on. In L, each role inherits permissions from its successor role and each role inherits users from its predecessor roles. This means that $users(R_{i_1}) \subset users(R_{i_2}) \subset \ldots \subset users(R_{i_L})$, which by our definition is a user chain and $perms(R_{i_L}) \subset perms(R_{i_{L-1}}) \subset \ldots \subset perms(R_{i_2}) \subset perms(R_{i_1})$ is a permission chain. Hence, L has to be either the longest user chain or longest permission chain or both in the RBAC policy. □

C Bound on the Number of Trials

Given a graph $G = (V, E)$, let $E_R \subseteq E$ denote the set of redundant edges that can be removed. Suppose we choose a subset of k vertices, and let G_S be the subgraph induced by these vertices.

Let X be the number of trials until all redundant edges are captured, and let X_i be the number of trials required to find exactly $i-1$ redundant edges. Then, $X = \sum_{i=1}^{|E_R|} X_i$. Probability of finding a new redundant edge in a given trial is

$$\Pr(\text{finding a new redundant edge}) = p_i \geq \frac{|E_R| - (i-1)}{\binom{k}{2}}.$$

Since X_i follows a geometric distribution, $\mathbb{E}[X_i] = \frac{1}{p_i} \leq \frac{\binom{k}{2}}{|E_R|-(i-1)}$. By linearity of expectation, we have

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{|E_R|} X_i\right] \leq \binom{k}{2} \sum_{i=1}^{|E_R|} \frac{1}{|E_R| - (i-1)} \leq k^2 \sum_{i=1}^{|E_R|} \frac{1}{|E_R| - (i-1)}$$
$$\leq k^2 \left(\log |E_R| + 1\right)$$

By Markov's inequality, $\Pr(X \geq t) \leq \frac{\mathbb{E}[X]}{t} \leq \frac{k^2 \log |E_R|}{t}$ where t is the number of trials. We can choose t in proportion to $\mathbb{E}[X]$ so that $\Pr(X \geq t)$ is sufficiently small.

D ILP Construction for Minimizing Number of Edges

We provide the mathematical formulation of the ILP based edge pruning in Role Hierarchies.

Mathematical formulation for Section 5.2.2.

$$\begin{aligned}
\text{minimize} \quad & \sum_{u \in U} \sum_{r \in R} x_{u,r} + \sum_{r \in R} \sum_{p \in P} y_{r,p} \\
\text{subject to} \quad & z_{u,e,x,p} \leq x_{u,e}, \quad \forall (u,p) \in UP,\ (e,x) \in \Pi_{u,p}, \\
& z_{u,e,x,p} \leq y_{x,p}, \quad \forall (u,p) \in UP,\ (e,x) \in \Pi_{u,p}, \\
& x_{u,r} \leq 0, \quad \forall u \in U,\ (u,r) \notin RH, \\
& y_{r,p} \leq 0, \quad \forall p \in P,\ (r,p) \notin RH, \\
& \sum_{(e,x) \in \Pi_{u,p}} z_{u,e,x,p} \geq 1, \quad \forall (u,p) \in UP, \\
& x_{u,r},\ y_{r,p},\ z_{u,e,x,p} \in \{0,1\} \quad \forall u \in U, p \in P, r \in R, (e,x) \in \Pi_{u,p}
\end{aligned} \tag{1}$$

Mathematical formulation for Section 5.2.3.

$$\begin{aligned}
\text{minimize} \quad & \sum_{\substack{(u,r) \in RH \\ u \in U, r \in R}} x_{u,r} + \sum_{\substack{(r,p) \in RH \\ p \in P, r \in R}} y_{r,p} + \sum_{\substack{(r,s) \in RH \\ r,s \in R}} z_{r,s} \\
\text{subject to} \quad & \left.\begin{aligned} P_{u,p,i} &\leq x_{u,r_1}, \\ P_{u,p,i} &\leq y_{r_k,p}, \\ P_{u,p,i} &\leq z_{r_j,r_{j+1}} \end{aligned}\right\} \begin{aligned} &\forall (u,p) \in UP, \\ &i^{\text{th}} \text{ path } r_1, \ldots, r_k \in \text{ paths } u \rightsquigarrow p \\ &1 \leq i < k \end{aligned} \\
& \sum_{\text{path } i \in \text{ paths } u \rightsquigarrow p} P_{u,p,i} \geq 1, \quad \forall (u,p) \in UP, \\
& x_{u,r},\ y_{r,p},\ z_{r,s} \in \{0,1\} \quad \forall u \in U, p \in P, r,s \in R \\
& P_{u,p,i} \in \{0,1\} \quad \forall u \in U, p \in P, \text{index } i \in \text{paths from } u \text{ to } p
\end{aligned} \tag{2}$$